

Credit Card Fraud Detection

Using Machine Learning & Advanced Techniques

Google Colab Link:

<https://colab.research.google.com/drive/1A2AGK69q7kjvezirfzkhXMLn7-LRHkNu?usp=sharing>

Report generated on 2026-05-01

1. Introduction

Credit card fraud detection is a critical problem in the financial industry due to the massive number of transactions processed daily. Fraudulent transactions are rare but highly impactful, making this a classic **imbalanced classification problem**.

Goal: Build a robust machine learning model capable of accurately detecting fraudulent transactions while minimizing false positives.

2. Objectives

- Detect fraudulent transactions effectively
- Maximize **Recall** to catch as many fraud cases as possible
- Maintain acceptable **Precision** to reduce false alarms
- Compare model performance with and without PCA
- Apply advanced techniques such as SMOTE and threshold tuning

3. Dataset Description

The dataset consists of anonymized credit card transactions:

Feature	Description
V1 to V28	Transformed using PCA for confidentiality
Time	Seconds between transactions
Amount	Transaction value
Class (Target)	0 = Normal 1 = Fraudulent

Key Challenge: The dataset is **highly imbalanced**, where fraudulent transactions represent ~0.17% of the data.

4. Data Preprocessing

The following preprocessing steps were applied:

- Checked and handled missing values (none found)
- Removed duplicate records

- Analyzed class distribution
- Split dataset into features (X) and target (y)
- Applied **train-test split with stratification**

Code: Data Loading & Initial Processing

```
# Import essential libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Load the dataset
data = pd.read_csv('creditcard.csv')

# Check for missing values
print(data.isnull().sum())

# Remove duplicate records
data = data.drop_duplicates()

# Split features and target
X = data.drop('Class', axis=1)
y = data['Class']

# Split data with stratification
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

5. Exploratory Data Analysis (EDA)

EDA was performed to understand the dataset:

- Visualized class imbalance
- Generated correlation heatmap
- Analyzed transaction time distribution
- Compared fraud vs normal transactions
- Used boxplots to detect outliers

Code: Visualization

```
import matplotlib.pyplot as plt
import seaborn as sns

# Visualize class distribution
plt.figure(figsize=(8, 6))
sns.countplot(data=data, x='Class', palette=['lightgreen', 'lightcoral'])
plt.title('Class Distribution: Normal vs Fraud')
plt.show()

# Correlation heatmap
plt.figure(figsize=(12, 10))
correlation_matrix = data.corr()
sns.heatmap(correlation_matrix, cmap='coolwarm', fmt='.2f')
plt.title('Feature Correlation Heatmap')
plt.show()
```

Insight: Fraud cases are extremely rare and show different patterns compared to normal transactions.

6. Feature Scaling

Applied **StandardScaler** to normalize feature values. Important for models like Logistic Regression.

```
# Apply StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

7. Handling Imbalanced Data (SMOTE)

To address class imbalance:

- Applied **SMOTE (Synthetic Minority Over-sampling Technique)**

- Generated synthetic samples for the minority class (fraud)

```
from imblearn.over_sampling import SMOTE # Apply SMOTE on training data only smote =  
SMOTE(random_state=42) X_train_resampled, y_train_resampled = smote.fit_resample(X_train_scaled,  
y_train) # Verify new balance print("Class distribution after SMOTE:")  
print(pd.Series(y_train_resampled).value_counts())
```

Result: Significantly improved model ability to learn fraud patterns.

8. Dimensionality Reduction (PCA)

Applied PCA with **10 components**. Reduced dimensionality while retaining most variance.

```
from sklearn.decomposition import PCA # Apply PCA pca = PCA(n_components=10) X_train_pca =  
pca.fit_transform(X_train_resampled) X_test_pca = pca.transform(X_test_scaled) # Display explained  
variance ratio print("Explained variance ratio:", pca.explained_variance_ratio_.sum())
```

Observation: PCA did not always improve performance, especially for tree-based models.

9. Models Implemented

9.1 Logistic Regression

Baseline model with class_weight='balanced'

Pros: High Recall, Fast training, Interpretable coefficients

Cons: Very low Precision (many false positives), Assumes linear relationship

```
from sklearn.linear_model import LogisticRegression # Create Logistic Regression model lr_model =
LogisticRegression( class_weight='balanced', random_state=42, max_iter=1000 )
lr_model.fit(X_train_resampled, y_train_resampled) # Predictions y_pred_lr =
lr_model.predict(X_test_scaled) y_prob_lr = lr_model.predict_proba(X_test_scaled)[: , 1]
```

9.2 Random Forest

Ensemble learning using multiple decision trees

Pros: Balanced Precision and Recall, Handles complex patterns, Feature importance ranking, Robust to outliers

Cons: Requires more computational resources, Can overfit with deep trees

```
from sklearn.ensemble import RandomForestClassifier # Create Random Forest model rf_model =
RandomForestClassifier( n_estimators=100, random_state=42, class_weight='balanced', max_depth=20 )
rf_model.fit(X_train_resampled, y_train_resampled) # Predictions y_pred_rf =
rf_model.predict(X_test_scaled) y_prob_rf = rf_model.predict_proba(X_test_scaled)[: , 1]
```

9.3 XGBoost

Gradient boosting model using sequential learning to correct errors

Pros: Strong performance on imbalanced data, High potential after tuning, Built-in regularization, Handles missing values

Cons: Prone to overfitting if not tuned properly, Hyperparameter tuning is complex

```
from xgboost import XGBClassifier # Create XGBoost model xgb_model = XGBClassifier( random_state=42,
scale_pos_weight=99, # Handle imbalance max_depth=6, learning_rate=0.1, n_estimators=200 )
xgb_model.fit(X_train_resampled, y_train_resampled) # Predictions y_pred_xgb =
xgb_model.predict(X_test_scaled) y_prob_xgb = xgb_model.predict_proba(X_test_scaled)[: , 1]
```

10. Model Evaluation

Models were evaluated using the following metrics:

Metric	Description
Accuracy	Overall correctness
Precision	Accuracy of fraud predictions
Recall	Fraud detection rate
F1 Score	Harmonic mean of Precision & Recall
ROC-AUC	Area under the ROC curve

Code: Evaluation Function

```
from sklearn.metrics import ( accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix, classification_report, roc_auc_score, roc_curve ) # Function to evaluate model
def evaluate_model(model, X_test, y_test, model_name):
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]
    print(f"=== {model_name} ===")
    print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
    print(f"Precision: {precision_score(y_test, y_pred):.4f}")
    print(f"Recall: {recall_score(y_test, y_pred):.4f}")
    print(f"F1 Score: {f1_score(y_test, y_pred):.4f}")
    print(f"ROC-AUC: {roc_auc_score(y_test, y_prob):.4f}")
    print(" Confusion Matrix:")
    print(confusion_matrix(y_test, y_pred))
    # Evaluate all models
    evaluate_model(lr_model, X_test_scaled, y_test, "Logistic Regression")
    evaluate_model(rf_model, X_test_scaled, y_test, "Random Forest")
    evaluate_model(xgb_model, X_test_scaled, y_test, "XGBoost")
```

Focus Metrics: Recall = Detect fraud cases | Precision = Reduce false positives

11. Threshold Tuning

Different thresholds were tested (0.3 to 0.7):

- Lower threshold = Higher Recall, lower Precision
- Higher threshold = Higher Precision, lower Recall

Code: Threshold Tuning

```
# Threshold tuning
thresholds = np.arange(0.3, 0.8, 0.1)
results = []
for threshold in thresholds:
    y_pred_thresh = (y_prob_rf >= threshold).astype(int)
    precision = precision_score(y_test, y_pred_thresh)
    recall = recall_score(y_test, y_pred_thresh)
    f1 = f1_score(y_test, y_pred_thresh)
    results.append([threshold, precision, recall, f1])
# Display results
results_df = pd.DataFrame(results, columns=['Threshold', 'Precision', 'Recall', 'F1'])
print(results_df)
# Find optimal threshold
optimal_idx = results_df['F1'].idxmax()
optimal_threshold = results_df.iloc[optimal_idx]['Threshold']
print(f"Optimal threshold: {optimal_threshold}")
```

Insight: Threshold tuning is crucial for balancing business needs between catching fraud and minimizing false alarms.

12. Hyperparameter Tuning

Used **GridSearchCV** for Random Forest:

Code: Grid Search

```
from sklearn.model_selection import GridSearchCV # Define parameter grid param_grid = {
'n_estimators': [100, 200, 500], 'max_depth': [None, 10, 20, 30], 'min_samples_split': [2, 5, 10],
'min_samples_leaf': [1, 2, 4] } # Grid Search with cross-validation grid_search = GridSearchCV(
RandomForestClassifier(class_weight='balanced', random_state=42), param_grid, cv=3, scoring='f1',
n_jobs=-1 ) grid_search.fit(X_train_resampled, y_train_resampled) print("Best parameters:",
grid_search.best_params_) print("Best F1 Score:", grid_search.best_score_) # Use best model
best_rf_model = grid_search.best_estimator_ evaluate_model(best_rf_model, X_test_scaled, y_test,
"Optimized Random Forest")
```

Result: Improved F1 Score and overall performance through systematic hyperparameter optimization.

13. Results Comparison

Without PCA:

Model	Performance	Notes
Random Forest	BEST	Most stable, balanced metrics
Logistic Regression	MODERATE	High recall but poor precision
XGBoost	GOOD	Strong after tuning

With PCA:

- Slight performance drop in tree-based models
- Better for linear models (Logistic Regression)
- Reduced training time but lost some feature information

14. Key Insights

- **Random Forest (without PCA)** performed best overall with balanced Precision and Recall
- **SMOTE** significantly improved fraud detection capability
- **PCA** is not always beneficial, especially for tree-based models
- **Threshold tuning** is essential for business-specific optimization
- **ROC-AUC** provides deeper understanding of model discrimination ability
- Class imbalance handling is more important than model complexity

15. Conclusion

This project successfully demonstrated a complete machine learning pipeline for fraud detection. The best-performing model was **Random Forest without PCA**, as it achieved a strong balance between precision and recall.

Key Takeaways:

- Handling class imbalance is critical for fraud detection
- Evaluation metrics must go beyond accuracy
- Model selection depends on the problem context and business requirements
- Ensemble methods (Random Forest, XGBoost) outperform single models
- Feature engineering and preprocessing are as important as model selection

16. Future Work

- Use **RandomizedSearchCV** for faster hyperparameter tuning
- Apply advanced techniques like **ADASYN** for better synthetic sampling
- Deploy model using **Streamlit** for interactive web application
- Explore deep learning approaches (**Neural Networks**, Autoencoders)
- Implement real-time fraud detection pipeline
- Use ensemble stacking for combining multiple models

17. Tools & Technologies

Tool	Purpose
Python	Programming Language
Pandas, NumPy	Data Manipulation
Scikit-learn	ML Models & Evaluation
XGBoost	Gradient Boosting
Matplotlib, Seaborn	Visualization
Imbalanced-learn	SMOTE & Sampling

18. Final Remark

Fraud detection is a real-world problem where model performance directly impacts financial security. Building a balanced and reliable model is essential for practical deployment. The choice between catching more fraud (high recall) versus reducing false alarms (high precision) should be guided by business cost analysis.

Remember: A fraud detection system is only as good as its ability to adapt to evolving fraud patterns. Continuous monitoring and model retraining are essential.

Project Link on Google Colab:

<https://colab.research.google.com/drive/1A2AGK69q7kjvezirfzkhXMLn7-LRHkNu?usp=sharing>